

Sistemas Operacionais

Prof. Rafael Obelheiro
rafael.obelheiro@udesc.br



Virtualização

Sumário

- 1 Introdução
- 2 Requisitos de um MMV
- 3 Implementação de MMVs
- 4 Suporte de hardware para virtualização
- 5 Virtualização em nível de SO

O que são máquinas virtuais (MVs)?

Definição formal (Popek & Goldberg, 1974)

Uma **máquina virtual** (MV) é vista como uma duplicata eficiente e isolada de uma máquina real. Essa abstração é construída por um **monitor de máquina virtual** (MMV).

- O MMV é também chamado de **hipervisor**
- Software executando em uma MV acredita ter total controle sobre a máquina
 - ▶ isso inclui um SO
- O MMV permite que múltiplas MVs coexistam no mesmo HW

Por que usar virtualização

- Utilização de recursos
 - ▶ multiplexar o HW existente entre várias aplicações/usuários
 - ▶ é possível migrar MVs para outro HW de forma transparente
- Uso e desenvolvimento de software
 - ▶ permite usar múltiplos SOs simultaneamente
 - ★ sem necessidade de dual boot
 - ▶ possibilita desenvolver SW de baixo nível (p.ex., SO) sem o incômodo usual (reboots)
- Várias outras possibilidades
 - ▶ depuração, emulação, segurança, tolerância a falhas
- Tema comum é que aplicações/serviços podem ser manipulados no nível de granularidade de uma máquina
 - ▶ versões específicas de SO, bibliotecas, aplicações, etc. empacotadas juntas

Sumário

- 1 Introdução
- 2 Requisitos de um MMV
- 3 Implementação de MMVs
- 4 Suporte de hardware para virtualização
- 5 Virtualização em nível de SO

Requisitos básicos de um MMV

- **Equivalência:** o MMV deve fornecer um ambiente de execução quase idêntico ao da máquina real original
 - exceções podem resultar de
 - ★ diferenças nos recursos disponíveis (memória, disco, etc.)
 - ★ dependências de temporização
 - ★ existência dos dispositivos de E/S necessários à aplicação
- **Controle de recursos:** o MMV deve ter controle total sobre o HW real
 - nenhum programa executando em uma MV deve ter acesso a recursos que não tenham sido explicitamente alocados à MV pelo MMV
 - a qualquer instante o MMV pode retirar recursos previamente alocados a uma MV
- **Eficiência:** grande parte das instruções de CPU em uma MV deve ser executada diretamente pela CPU real
 - instruções que não puderem ser executadas pela CPU real devem ser interpretadas e traduzidas em ações equivalentes pelo MMV

Requisitos derivados

- **Isolamento:** aplicações dentro de uma MV não podem interagir diretamente
 - (a) com outras máquinas virtuais
 - (b) com o hipervisor
 - (c) com o sistema real hospedeiro
 - todas as interações entre entidades dentro de uma MV e o mundo exterior devem ser mediadas pelo hipervisor
- **Introspecção:** o MMV tem acesso e controle sobre todas as informações do estado interno da MV
 - registradores de CPU, conteúdo da memória, eventos, ...
- **Recursividade:** alguns MMVs permitem executar outro MMV dentro de uma máquina virtual

Teorema de Popek & Goldberg (1)

Teorema de Popek & Goldberg (1974)

Para qualquer computador convencional de 3ª geração, um hipervisor pode ser construído se o conjunto de instruções sensíveis desse computador for um subconjunto de seu conjunto de instruções privilegiadas

- **Computador de 3ª geração:** arquitetura de Von Neumann, com suporte a memória virtual e processador com ≥ 2 modos de operação (usuário e supervisor)
- **Instruções sensíveis**
 - instruções que podem ler ou alterar o estado da CPU
 - ★ acessam registradores que armazenam o estado atual da CPU real
 - instruções cuja semântica depende do modo de operação da CPU
- **Instruções privilegiadas:** só podem ser executadas com a CPU em modo supervisor
 - caso código não privilegiado tente executar uma instrução privilegiada, uma exceção (interrupção) deve ser gerada, ativando uma rotina de tratamento previamente especificada pelo núcleo do sistema real $\rightarrow$ *trap*

Teorema de Popek & Goldberg (2)

Teorema de Popek & Goldberg (1974)

Para qualquer computador convencional de 3ª geração, um hipervisor pode ser construído se o conjunto de instruções sensíveis desse computador for um subconjunto de seu conjunto de instruções privilegiadas

- Consequência: uma CPU que satisfaça o teorema de Popek & Goldberg é virtualizável
- Nem todas as CPUs satisfazem o teorema
 - x86 é um exemplo
 - ★ Pentium IV tinha 18 instruções sensíveis que não eram privilegiadas

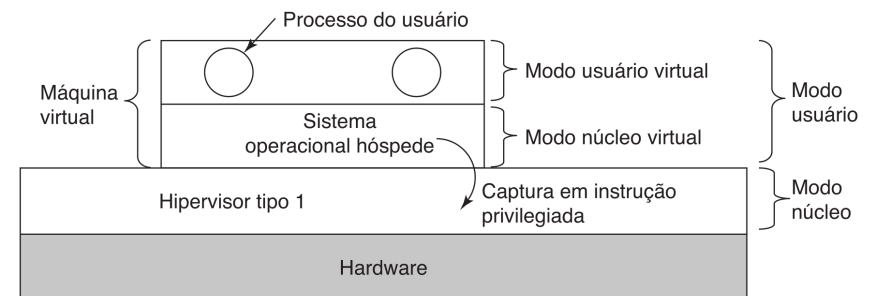
Sumário

- 1 Introdução
- 2 Requisitos de um MMV
- 3 Implementação de MMVs
- 4 Suporte de hardware para virtualização
- 5 Virtualização em nível de SO

Modelo aproximado de um MMV

- O MMV executa com o máximo privilégio na máquina real
 - o SO em uma MV executa em um nível inferior de privilégio
 - o MMV multiplexa os recursos entre as MVs
- Deseja-se que o código do SO em uma MV execute diretamente na CPU
 - como se o SO fosse um processo para o MMV
 - que partes do código do SO podem executar diretamente, e que partes podem causar problemas?
- Idealmente, instruções privilegiadas deveriam ser capturadas e tratadas pelo MMV → **trap-and-emulate**
 - instruções não privilegiadas executam diretamente na CPU

Trap-and-emulate

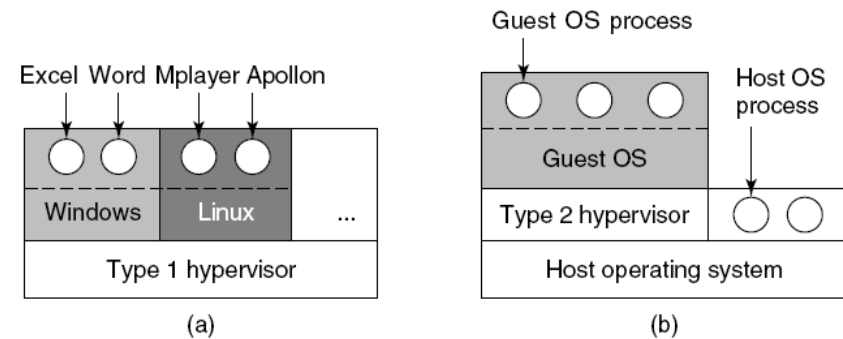


- Quando o SO hospede executa uma instrução privilegiada, o processador gera um **trap**
 - uma interrupção de software, tratada por código em modo núcleo (real)
- O **trap** é atendido pelo MMV, que **emula** a instrução privilegiada
 - analisa o que causou o **trap**, e executa instruções que produzem o efeito desejado na MV

Virtualizando o x86

- A facilidade para virtualizar depende da arquitetura
- A arquitetura x86 é complicada de virtualizar, mas é ao mesmo tempo a mais popular
- Problemas do x86
 - ▶ eventos não virtualizáveis
 - ★ instruções como `popf` não causam *traps* quando não modificam flags do sistema
 - ▶ TLB gerenciada pelo HW
 - ★ o MMV não pode tratar um *miss* na TLB
 - ▶ entradas na TLB não possuem tags
 - ★ toda a TLB precisa ser invalidada a cada chaveamento de contexto ⇒ questão de desempenho
- Com a popularização, AMD e Intel adicionaram suporte a virtualização em seus processadores

Hipervisores tipo 1 e tipo 2



- (a) Tipo 1: executa direto sobre o HW
 - ▶ Xen, VMware ESX/ESXi, IBM z/VM
- (b) Tipo 2: executa em um SO hospedeiro → MMV é um processo, depende do hospedeiro para acesso ao HW
 - ▶ VirtualBox, VMware Workstation

O que precisa ser virtualizado

- Exatamente o que se pode esperar
 - ▶ CPU
 - ▶ eventos (interrupções e exceções)
 - ▶ memória
 - ▶ dispositivos de E/S
- Isso não duplica a funcionalidade do SO no hipervisor?
 - ▶ sim e não
 - ▶ as abordagens são similares
 - ▶ o MMV é mais simples, mas implementa uma interface diferente

Virtualizando instruções privilegiadas

- O SO não pode mais executar instruções privilegiadas diretamente
 - ▶ memória virtual, interrupções, dispositivos de E/S, etc.
- Instruções que geram exceções podem ser capturadas e tratadas pelo hipervisor
- Instruções que não geram exceções requerem uma abordagem diferente
 - ▶ **tradução binária dinâmica**: blocos de código que contêm instruções privilegiadas são reescritos *on-the-fly* para serem emulados pelo MMV
 - ★ VMware mostrou que é possível implementar com overhead razoável
 - ▶ **paravirtualização**: partes do SO que exigem instruções privilegiadas são reescritas para fazer chamadas ao MMV
 - ★ *hypercalls*

Virtualizando a CPU

- O hipervisor precisa multiplexar as CPUs físicas entre as MVs
- A solução usada é alocar fatias de tempo para as MVs nas CPUs
 - um SO convidado escalona seus processos durante a fatia de tempo atribuída à sua MV
- Tipicamente é usado um escalonador simples (round-robin)
 - se um quantum não for usado (no todo ou em parte), ele pode ser alocado a outra MV
 - ★ requer identificar quando o sistema convidado está ocioso

Virtualizando eventos e E/S

- O hipervisor recebe interrupções e exceções de hardware
 - esses eventos precisam ser encaminhados para a MV apropriada
 - em alguns casos, os eventos são remapeados pelo MMV
 - ★ o HW gera a interrupção X, e o MMV sinaliza a intr Y na MV A
 - MMV também precisa emular os registradores de eventos
- SOs não podem mais interagir diretamente com os dispositivos de E/S
 - VMware Workstation: apenas dispositivos genéricos (hospedados)
 - ★ hipervisor expõe dispositivos virtuais conhecidos
 - ⇒ p.ex., interface Ethernet PCNet com chipset AMD Lance
 - ★ SO na VM usa um driver compatível com esse dispositivo
 - ★ driver interage com o MMV para repassar requisições para um driver real (p.ex., driver do SO hospedeiro)
 - VMware ESX Server: drivers executam diretamente no MMV
 - ★ lista de compatibilidade de HW é geralmente menor que SO tradicional

Virtualizando memória

- SOs pressupõem que têm controle total sobre a memória
 - para gerenciar: toda a memória pertence ao SO
 - para mapear: o SO pode mapear qualquer página virtual em qualquer página física
- Um hipervisor particiona a memória entre as VMs
 - MMV precisa atribuir páginas de hardware a MVs
 - MMV precisa controlar os mapeamentos para garantir isolamento
 - ★ não pode deixar um SO mapear uma página virtual em qualquer página física
 - ★ SO só pode mapear uma PV em uma PF alocada pelo MMV
- TLBs gerenciadas por HW dificultam a tarefa
 - quando ocorre um *miss*, o HW percorre as tabelas de páginas na memória automaticamente para resolvê-lo
 - como resultado, o MMV precisa controlar o acesso dos SOs convidados às tabelas de páginas

Tabelas de páginas sombra (1)

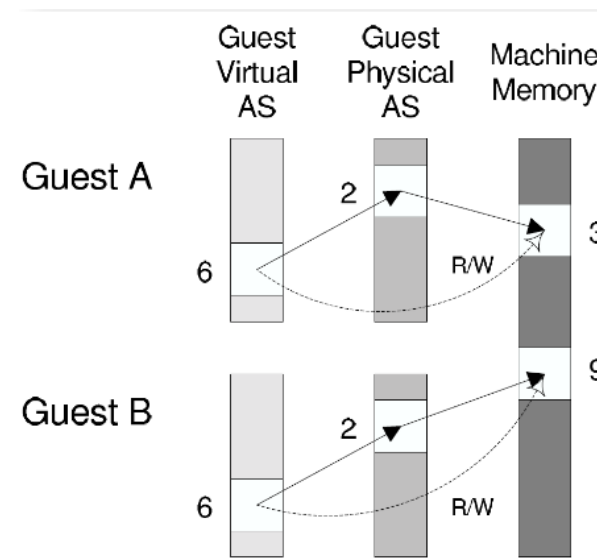
- Três abstrações de memória
 - **memória de máquina:** memória física real
 - **memória física:** abstração da memória real que é gerenciada por um SO convidado
 - ★ se o MMV aloca 1 GB para uma MV, o SO pensa que a máquina possui 1 GB de memória física contígua
 - ★ memória de máquina subjacente não precisa ser contígua
 - **memória virtual:** espaços de endereçamento virtual tradicionais usados pelos processos
- Em cada MV, o SO cria e gerencia tabelas de páginas para seus espaços de endereçamento virtuais sem modificações
 - essas tabelas não são usadas pela MMU, porém

Tabelas de páginas sombra (2)

- O hipervisor cria e gerencia tabelas de páginas que mapeiam páginas virtuais diretamente em páginas de máquina → **tabelas de páginas sombra** (*shadow page tables*)
 - carregadas na MMU nas trocas de contexto
- O hipervisor precisa manter suas tabelas V→M consistentes com as alterações feitas pelo SO em suas tabelas V→F
 - MMV mapeia as tabelas de páginas do SO com proteção contra escrita
 - quando o SO modifica uma tabela de páginas, ocorre um *trap* para o MMV
 - o MMV escreve na TP sombra e na TP do SO e retorna
 - também conhecido como *tracing* de memória
 - ★ mais overhead...

© 2025 Rafael Obelheiro (DCC/UDESC) Virtualização SOP 21/36

Tabelas de páginas sombra (3)



© 2025 Rafael Obelheiro (DCC/UDESC) Virtualização SOP 22/36

Alocação de memória

- As políticas de alocação de memória de hardware dos MMVs tendem a ser simples
 - estática: uma MV recebe 1 GB de RAM por toda eternidade
 - a alocação não é ajustada dinamicamente em função da carga
 - ★ muitos SOs não estão preparados para lidar com mudanças na memória física disponível
 - sem swapping
- Uma alocação mais sofisticada usa *overcommit* com driver de balão
 - driver de balão executa dentro do SO convidado
 - páginas alocadas ao driver são liberadas no hospedeiro, que pode alocá-las a outras MVs
 - ★ driver informa ao hipervisor suas faixas de endereços
 - ★ balão “rouba” páginas dos caches de disco e páginas
 - balão pode ser inflado (esvaziado) para reduzir (aumentar) a memória efetivamente alocada à MV
- O MMV pode mapear páginas virtuais idênticas de diversas MVs nas mesmas páginas físicas usando *copy-on-write*
 - **deduplicação de memória**

© 2025 Rafael Obelheiro (DCC/UDESC) Virtualização SOP 23/36

Sumário

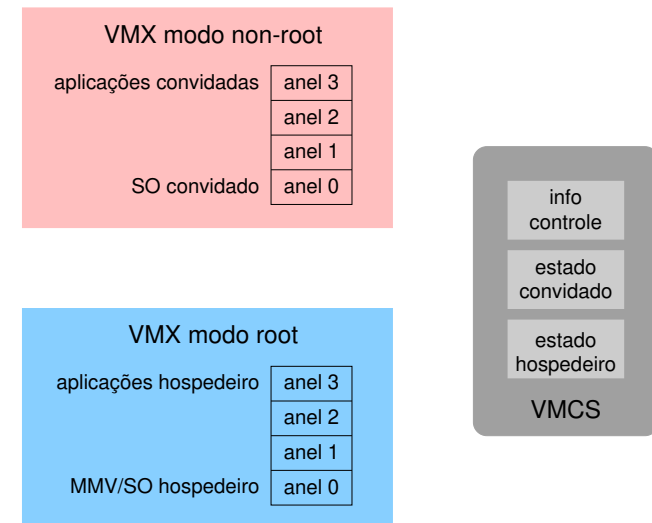
- 1 Introdução
- 2 Requisitos de um MMV
- 3 Implementação de MMVs
- 4 Suporte de hardware para virtualização
- 5 Virtualização em nível de SO

© 2025 Rafael Obelheiro (DCC/UDESC) Virtualização SOP 24/36

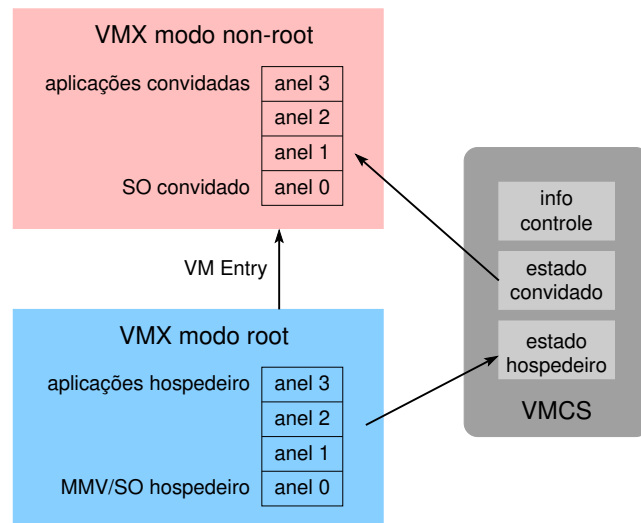
Suporte de hardware (1)

- Intel e AMD implementaram suporte a virtualização em seus processadores x86
 - Intel VT-x, AMD-V
- Modelo de execução direta
 - introduz dois modos de CPU
 - ★ VMX root: para executar MMV ou SO hospedeiro
 - ★ VMX não root: para executar MVs (modo convidado)
 - ★ cada modo possui seus próprios níveis de privilégio (anéis)
 - ★ possibilita execução direta do código do SO convidado, incluindo instruções privilegiadas
 - *virtual machine control structure* (VMCS)
 - ★ controla quais instruções causam *traps*, registra informações para que o hipervisor possa tratar os *traps*, mantém estado da CPU para os dois modos (→ troca de contexto)
 - novas instruções *vmlaunch/vmresume* entram no modo convidado, executam código na MV
 - quando ocorre um *trap*, a CPU executa uma nova instrução *vmexit*
 - ★ entra no hipervisor, que emula a operação

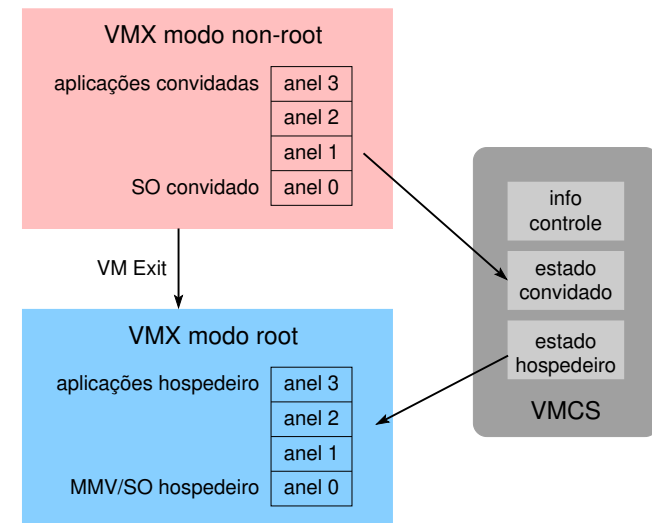
Intel VT-x



Intel VT-x: VM Entry



Intel VT-x: VM Exit

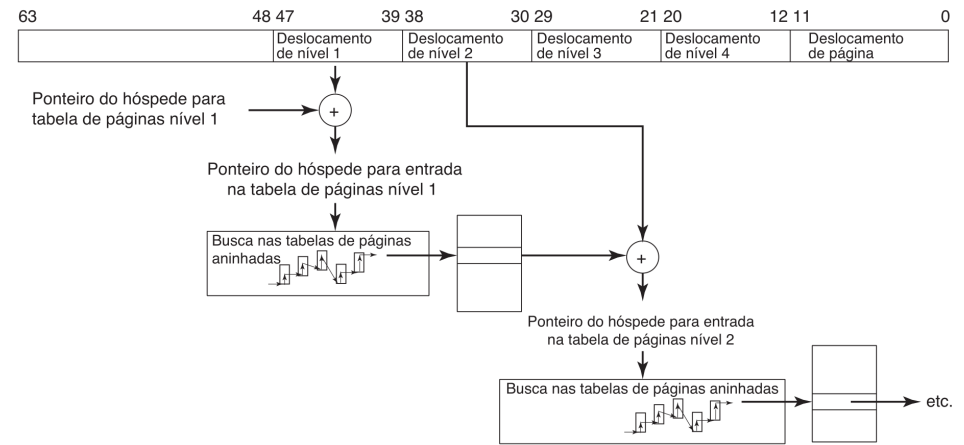


Suporte de hardware: memória

- Tabelas de páginas estendidas (Intel) e aninhadas (AMD)

- ▶ tabelas de páginas originais mapeiam páginas virtuais em físicas (do convidado)
 - ★ gerenciadas pelo SO na VM, sem necessidade de alterações no código
 - ★ sem necessidade de efetuar um *trap* para o MMV quando o SO atualiza suas TPs
- ▶ novas tabelas mapeiam páginas físicas a páginas de máquina
 - ★ gerenciadas pelo MMV
- ▶ entradas na TLB agora possuem tags com identificadores de processadores virtuais (VPID)
 - ★ cada MV recebe um VPID
 - ★ não precisa mais invalidar toda a TLB em uma transição MV↔MMV

Tabelas de páginas estendidas/aninhadas



Suporte de hardware: E/S

- Transferências usando DMA devem ser restritas a páginas alocadas a uma dada MV

- ▶ restrições implementadas via IOMMU
 - ★ suporte a remapeamento de interrupções e tradução de endereços para DMA

Sumário

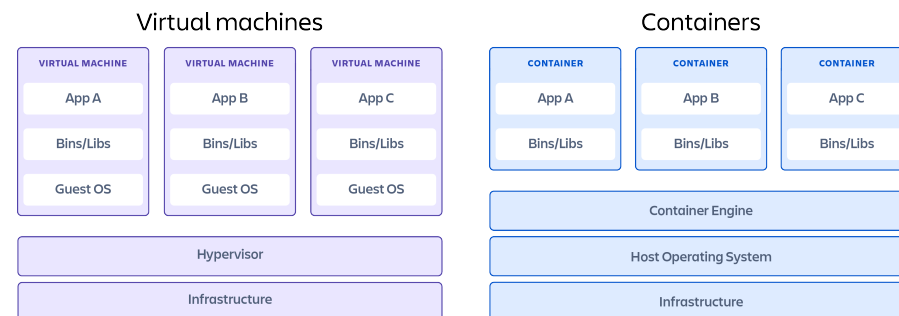
- 1 Introdução
- 2 Requisitos de um MMV
- 3 Implementação de MMVs
- 4 Suporte de hardware para virtualização
- 5 Virtualização em nível de SO

Virtualização em nível de SO (contêineres)

- Um **contêiner** é um grupo de processos no Linux
- cgroups (*control groups*) limitam os recursos de CPU, memória, rede e disco que um contêiner pode usar
 - ▶ também permitem atribuir prioridades
 - ★ p.ex., um grupo pode receber uma fração maior da CPU ou da largura de banda do disco
- Espaços de nomes (*namespaces*) limitam os recursos que os contêineres enxergam → isolamento
 - ▶ exemplos
 - ★ o EN `mnt` controla que partes do sistema de arquivos são visíveis
 - ★ o EN `pid` isola os PIDs que um contêiner pode manipular
 - ★ o EN `net` controla quais interfaces e regras de firewall um contêiner pode usar

© 2025 Rafael Obelheiro (DCC/UDESC) Virtualização SOP 33/36

Máquinas virtuais vs contêineres (1)



Fonte: <https://www.atlassian.com/microservices/cloud-computing/containers-vs-vms>

© 2025 Rafael Obelheiro (DCC/UDESC) Virtualização SOP 34/36

Máquinas virtuais vs contêineres (2)

- Vantagens de contêineres
 - ▶ não é necessário traduzir ou reescrever código convidado
 - ▶ overhead menor
 - ★ evita trocas de contexto entre aplicações convidadas, SO convidado e hipervisor
 - ★ dispensa tabelas de páginas sombra/estendidas/aninhadas
 - ★ não é necessário bootar ou gastar memória com um SO inteiro para executar uma aplicação
 - ▶ snapshots são menores do que com MVs tradicionais
 - ★ não é necessário salvar o estado do SO
- Desvantagens de contêineres
 - ▶ aplicações precisam ser compatíveis com o SO hospedeiro
 - ★ não é possível executar um SO convidado diferente
 - ▶ nível de isolamento é menor

© 2025 Rafael Obelheiro (DCC/UDESC) Virtualização SOP 35/36

Bibliografia Básica

- 📄 **Andrew S. Tanenbaum e Herbert Bos.**
Sistemas Operacionais Modernos, 4ª Edição. Capítulo 7.
Pearson Prentice Hall, 2016.
- 📄 **Carlos A. Maziero.**
Sistemas Operacionais: Conceitos e Mecanismos. Capítulos 31 a 34.
Editora da UFPR, 2019.
<http://wiki.inf.ufpr.br/maziero/doku.php?id=socm:start>

© 2025 Rafael Obelheiro (DCC/UDESC) Virtualização SOP 36/36